Software Requirements Specification

System: KodBTW — Unified Coding Identity & Campus Coding Intelligence Platform

Version: 1.0, evidence-based description of repository snapshot 1d45df1

Format: IEEE-style requirements specification adapted to the actual implementation; no institutional SRS template was present in the supplied material.

1. Introduction

1.1 Purpose

This specification describes the externally visible requirements and constraints supported by the checked-out KodBTW application. It is intended for course review, maintenance, and traceability. It records current behavior and labels proposed behavior separately.

1.2 Scope

KodBTW is a web application for maintaining a user account and profile, associating coding-platform handles, synchronizing supported public statistics, viewing persisted platform snapshots and derived analytics, consulting a leaderboard, and sharing an opt-in public developer profile. The current implementation is a single-user-facing system; it is not a campus administration or mentor-monitoring product.

1.3 Definitions and abbreviations

Term	Meaning
API	Application Programming Interface
JWT	JSON Web Token carried as a bearer token
Snapshot	Persisted platform-stat observation keyed by user, platform, and date
REAL source	A snapshot tagged with a source such as LEETCODE_REAL or CODEFORCES_REAL
MOCK	Synthetic/test data source; not presented as verified live platform statistics
SRS	Software Requirements Specification

1.4 References

• Project architecture and platform-data notes in docs/ARCHITECTURE.md, docs/CODEFORCES_INTEGRATION.md, and docs/platform-data-acquisition.md.

• Flyway migrations V1-V7 in backend/src/main/resources/db/migration/.

• REST controllers in backend/src/main/java/com/kodbtw/controller/.

• IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications (historical format reference; this document is adapted, not a claim of formal compliance).

2. Overall description

2.1 Product perspective

The React 18 / TypeScript / Vite frontend calls a Spring Boot 3.3 REST API. The API validates requests, applies JWT authentication and owner scoping, calls platform adapters, persists data through Spring Data JPA, and uses MySQL 8 with Flyway-managed schema migrations. Codeforces and LeetCode are live integrations; the other three catalogued platforms are connectable but have no live statistics source.

2.2 Product functions

• Register, log in, and request the authenticated user's identity.

• Read and edit the user's KodBTW profile.

• Create, list, read, update, and delete the user's coding-platform accounts.

• Read current persisted platform statistics, request a sync, and inspect sync status for an owned account.

• View dashboard totals, analytics, historical snapshots, insights, activity, and contest intelligence.

• Read global/college leaderboard pages and the current user's rank; run an admin-authorized cache rebuild.

• Read a public developer profile by profile username.

2.3 User classes

• Registered user: owns a KodBTW account, profile, platform-account records, and authenticated dashboard data.

• Public visitor: may open the landing page and a public profile endpoint/page.

• Administrator authority: may call the protected manual leaderboard rebuild route. The source does not implement a distinct mentor or college-administrator application role workflow.

2.4 Operating environment

• Backend: Java 21, Spring Boot 3.3, Maven, MySQL 8.

• Frontend: Node.js 18+ and npm, React 18, TypeScript, Vite.

• Deployment notes describe a containerized backend and static frontend, but no production host, deployed URL, database, or deployment has been verified.

2.5 Constraints, assumptions, and dependencies

• DB_URL, DB_USERNAME, DB_PASSWORD, and JWT_SECRET must be provided at runtime. Secrets are not expected in Vite build variables.

• MySQL schema evolution is managed by Flyway migrations V1-V7; Hibernate validates the schema and does not create/update tables.

• CORS production origins must be explicitly configured; wildcard origins are rejected by production configuration.

• Statistics depend on upstream platform availability and exposed fields. Null means unavailable or not supplied; the system must not infer values from absent fields.

• The LeetCode query uses a publicly reachable but undocumented GraphQL endpoint and may change. Its contractual stability and terms are not guaranteed by the code.

• The Codeforces user.info endpoint provides only the subset mapped by the adapter; it does not supply solved counts or history.

3. External interface requirements

3.1 User interface

Routes found in frontend/src/App.tsx are /, /login, /register, /u/:username, /dashboard, /analytics, /contests, /leaderboard, /profile, and /coding-profiles. The last six routes are guarded by the frontend's protected-route component. Actual visual behavior has not been validated in a running instance in this environment.

3.2 REST interfaces

Requirement group	Implemented API surface	Access
Authentication	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me	Register/login public; identity requires bearer JWT
Profile	GET /api/profile, PUT /api/profile	Authenticated, current user
Public profile	GET /api/public/profiles/{username}	Public
Dashboard	GET /api/dashboard/stats, /analytics, /history, /insights, /activity, /contests	Authenticated, current user
Platform accounts	/api/platform-accounts/	Authenticated; service checks record ownership
Leaderboard	GET /api/leaderboard, /college, /me; POST /api/leaderboard/sync	Authenticated reads; sync requires ADMIN authority
Health	GET /api/health	Public process-liveness response

3.3 Software interfaces

The backend uses JDBC/MySQL through JPA and Flyway. Platform adapter clients issue requests to LeetCode and Codeforces. No platform credentials or user passwords are collected for those public profile lookups.

4. Functional requirements

ID	Requirement	Implementation / verification mapping	Status
FR-01	The system shall accept a valid registration request and create a user account.	AuthController, UserService, RegisterRequest; AuthControllerTest	Implemented; test not run here
FR-02	The system shall authenticate a registered user and return a bearer JWT on valid credentials.	AuthController, AuthService, JwtService; AuthControllerTest	Implemented; test not run here
FR-03	The system shall expose the identity of the current authenticated user.	GET /api/auth/me; AuthControllerTest	Implemented; test not run here
FR-04	The system shall allow a user to read and update their own KodBTW profile.	ProfileController, ProfileService; profile controller/service tests	Implemented; test not run here
FR-05	The system shall allow an authenticated user to create and list their own platform-account records.	PlatformAccountController, PlatformAccountService; controller tests	Implemented; test not run here
FR-06	The system shall permit a user to read, update, or remove only an account they own.	PlatformAccountService; PlatformAccountControllerTest	Implemented; test not run here

ID	Requirement	Implementation / verification mapping	Status
FR-07	The system shall expose live statistics sync only for platforms whose hasLiveStatsSource() is true.	Platform.hasLiveStatsSource, PlatformSyncService; sync/adapter tests	Implemented for LeetCode and Codeforces; tests not run here
FR-08	The system shall persist supported sync observations as dated platform snapshots and preserve source tags.	PlatformSyncPersistenceService, PlatformStatSnapshot, V4/V6/V7; service tests	Implemented; tests not run here
FR-09	The system shall return unavailable values as null and shall not fabricate missing platform metrics.	Platform adapters, stats services, platform-data note; adapter tests	Implemented in documented mappings; tests not run here
FR-10	The system shall return the authenticated user's dashboard totals and analytics from eligible persisted data.	DashboardController, PlatformStatsService, analytics services; controller/service tests	Implemented; tests not run here
FR-11	The system shall provide historical views based on stored observations rather than synthesized history.	DashboardHistoryService; history tests	Implemented; tests not run here
FR-12	The system shall provide activity and contest-intelligence responses from persisted data.	DashboardActivityContestService; service tests	Implemented; no independent contest-feed ingestion found
FR-13	The system shall provide authenticated global and college-filtered leaderboard reads with pagination and supported sort/filter options.	LeaderboardController, LeaderboardService; controller/service tests	Implemented; tests not run here
FR-14	The system shall restrict the manual leaderboard rebuild endpoint to a principal with ADMIN authority.	SecurityConfig, LeaderboardController; security/controller tests	Implemented; tests not run here
FR-15	The system shall provide a public profile by unique profile username, returning the public-profile response rather than requiring a JWT.	PublicProfileController, PublicProfileService, V5; tests	Implemented; tests not run here
FR-16	The system shall provide a public liveness endpoint.	HealthController; health test	Implemented; tests not run here

5. Non-functional requirements

ID	Requirement	Evidence / verification	Status
NFR-01	Password authentication shall compare encoded passwords; authentication tokens shall use a configured signing secret.	AuthService, JwtService, environment configuration	Implemented; no live security audit performed
NFR-02	Authenticated resources shall use stateless bearer-token security.	SecurityConfig	Implemented

ID	Requirement	Evidence / verification	Status
NFR-03	User-specific account and dashboard operations shall be scoped to the authenticated owner.	Service/controller ownership checks and tests	Implemented in source; test suite not executed
NFR-04	The production configuration shall not contain committed database credentials or a fallback JWT secret.	application.yml, application-prod.yml, .env.example	Implemented as configuration policy
NFR-05	Schema changes shall be versioned, validated, and applied in order; automatic Hibernate schema mutation and Flyway clean/baseline bypass shall be disabled.	application.yml, migrations V1-V7	Implemented
NFR-06	API errors shall not disclose stack traces or binding details in the HTTP response.	application.yml server error settings	Implemented by configuration; runtime not verified
NFR-07	Production cross-origin access shall use configured exact origins.	CorsConfig, CorsConfigTest	Implemented in source; test not run
NFR-08	Historical/analytics output shall be traceable to persisted observations and source tags.	Snapshot schema and read services	Implemented in source; data-backed runtime not verified
NFR-09	The system should remain operable with absent upstream metrics by representing unsupported values as null.	DTO/adapters and tests	Implemented in source; test not run

6. Data requirements and validation rules

• Email is unique in users; profile username is unique in profiles; each user has at most one profile; (user_id, platform) is unique for platform accounts; (user_id, platform, snapshot_date) is unique for snapshots.

• Profile fields and platform account URLs are length-limited by entity/schema mappings. Request DTOs use Jakarta validation.

• Foreign keys cascade deletion from a user to their profile, linked accounts, snapshots, and leaderboard cache.

• Snapshot metrics are nullable because upstream sources provide different subsets. Data source and observation/snapshot timestamps are retained.

• A user may link accounts across the five platform enum values. Connecting an account does not imply that live sync is available or that account ownership has been verified; the verified field defaults false and no verification workflow is asserted.

7. Security and privacy requirements

• The API shall require JWT authentication except for registration/login, health, and public profile reads.

• Account operations shall verify ownership. The public profile shall expose sanitized public-profile DTOs, not user credentials.

• Secrets shall be injected through runtime environment/secret management and not compiled into client assets.

• The application must not claim that a public webpage or undocumented endpoint constitutes permission for automated collection. The current platform-data note explicitly keeps three integrations unavailable; LeetCode's

undocumented query remains a dependency and terms-review limitation.

• The code does not implement a separate mentor or college-admin role/consent/monitoring workflow.

8. Use cases

UC-01 Register and sign in

Actor: Visitor. Preconditions: API available. Main flow: visitor submits registration fields; API validates and creates the account; visitor submits credentials; API verifies them and returns JWT and user summary. Alternatives: validation, duplicate email, or invalid credentials return errors. Result: authenticated frontend can access guarded routes.

UC-02 Connect and synchronize a coding account

Actor: Registered user. Preconditions: valid JWT. Main flow: user adds platform and handle; API stores an owned platform-account record; on manual sync, the service checks ownership and platform support, calls the adapter, persists a source-tagged snapshot, and updates sync status. Alternatives: unsupported platform reports live sync unavailable; upstream/not-found/rate-limit failures are returned as categorized errors. Result: current stats are available only if supported persisted data exists.

UC-03 Review progress

Actor: Registered user. Preconditions: JWT and available snapshots. Main flow: frontend calls dashboard, analytics, history, activity, and contest endpoints; services read user-scoped snapshots and return computed views. Alternative: missing observations remain absent/null; no historical values are fabricated. Result: current and historical views reflect stored source data.

UC-04 Share public profile

Actor: Registered user and public visitor. Preconditions: profile username exists and the user has populated public profile data. Main flow: user uses the share UI to obtain the /u/{username} page; visitor page fetches public profile data without JWT. Result: public DTO-backed profile is displayed. Visibility-setting semantics require code review before claiming additional controls.

9. Acceptance criteria

Acceptance criteria are linked to source and test names above. The current delivery is not marked test-verified because the available environment lacked Java/JDK and npm. Before release, run the backend Maven suite, frontend production build, migration validation against MySQL 8, and authenticated/public route checks using a configured development environment.

10. Requirements traceability matrix

The functional and non-functional requirement tables are the traceability matrix. They identify source files, test classes when present, and the current verification status. “Test not run here” means a corresponding test source exists or the requirement is covered by source/configuration review; it is not a test result.

11. Explicitly proposed but out of current scope

The proposal lists mentor/college-admin dashboards, campus registration, student monitoring, inactivity detection, DSA topic-gap analysis, workshop suggestions, AI suggestions, account verification, an upcoming-contest feed, and profile analytics beyond the current source-backed behavior. These are not acceptance requirements for the current implementation and must not be represented as delivered.